

调研 5/14晚会

4天调研总目标

最终需要产出一份统一报告，回答5个核心问题：

1. 数据从哪里来？
2. 知识图谱怎么建？
3. 简历/JD怎么结构化？
4. 推荐算法怎么做？
5. 最终系统怎么展示？

建议最终报告结构如下：

代码块

- 1 1. 项目背景与目标
- 2 2. 数据源与知识图谱设计
- 3 3. NLP画像抽取方案
- 4 4. 推荐算法方案
- 5 5. 系统架构与展示方案
- 6 6. 4人分工与后续开发计划
- 7 7. 风险与备选方案

A：数据与知识图谱调研内容压缩版 whY

A 的4天核心任务

A 找数据源，确保哪些是能下载的，最好试着下一点

我们准备用什么数据？数据字段能不能支撑知识图谱和推荐系统？

A 第1天：数据源快速调研

重点找3类数据：

优先级	数据源	用途
最高	阿里天池智联招聘人岗匹配数据	中文人岗推荐主数据
高	Kaggle Job Recommendation / Resume Matching	英文备选数据
中	HuggingFace job-skill / resume-job matching 数据	技能图谱和语义匹配辅助

A 需要整理成表：

数据集	是否可下载	字段	规模	是否适合本项目	风险
-----	-------	----	----	---------	----

重点不是下载完所有数据，而是确认：哪个数据可以作为主数据，哪个可以作为备选。

A 第2天：设计数据结构与知识图谱 Schema

A 需要确定实体和关系。

建议采用最小知识图谱 Schema：

实体

代码块

```
1  Candidate
2  Job
3  Skill
4  Company
5  City
6  Industry
7  Education
8  Experience
```

关系

代码块

```
1  Candidate - HAS_SKILL - Skill
2  Job - REQUIRES_SKILL - Skill
```

- 3 Job - LOCATED_IN - City
- 4 Job - BELONGS_TO - Industry
- 5 Company - PUBLISHES - Job
- 6 Candidate - APPLIED_TO - Job
- 7 Candidate - MATCHES - Job

A 第3天：画图谱结构图

A 需要给报告提供一张图，展示：

代码块

- 1 候选人 → 技能 ← 岗位
- 2 候选人 → 投递 → 岗位
- 3 岗位 → 公司
- 4 岗位 → 城市
- 5 岗位 → 行业
- 6 岗位 → 学历要求

可以用 draw.io、PPT、ProcessOn、XMind 画，不需要真的导入 Neo4j。

A 第4天：整理报告内容

A 最终提交：

代码块

- 1 1. 数据源对比表
- 2 2. 选定主数据集与备选数据集
- 3 3. 数据字段设计
- 4 4. 知识图谱实体关系设计
- 5 5. Neo4j是否可行
- 6 6. 数据风险与解决方案

B：NLP与画像抽取调研内容压缩版 郭靖宇

B 的4天核心任务

B 需要回答：

B 第1天：确定抽取字段

B 不需要一开始做复杂NER，只需要明确要抽什么。

简历抽取字段

代码块

- 1 学历
- 2 专业
- 3 工作/项目经验
- 4 技能
- 5 项目经历
- 6 证书
- 7 期望岗位
- 8 期望城市

JD抽取字段

代码块

- 1 岗位名称
- 2 岗位职责
- 3 必需技能
- 4 加分技能
- 5 学历要求
- 6 经验要求
- 7 工作城市
- 8 行业方向
- 9 薪资范围

B 第2天：调研抽取方法

重点比较4种方法即可：

方法	优点	缺点	是否采用
正则表达式	快速、可控	泛化弱	用于学历、经验、薪资
技能词典匹配	简单稳定	依赖词典	用于技能抽取
BERT-NER	效果更好	训练成本高	作为扩展
大模型抽取	效果好、开发快	成本和稳定性问题	可用于小样本演示

建议你们调研结论写成：

本项目初期采用“规则 + 技能词典 + 关键词匹配”的轻量方案，后续可扩展为 BERT-NER 或大模型辅助抽取。

这样最稳。

B 第3天：设计画像 JSON 格式

B 要给 A 和 C 一个标准输出格式。

候选人画像

代码块

```
1  {
2    "candidate_id": "C001",
3    "education": "本科",
4    "major": "计算机科学与技术",
5    "experience_years": 1,
6    "skills": ["Python", "PyTorch", "SQL", "Linux"],
7    "project_domains": ["深度学习", "推荐系统"],
8    "expected_city": "广州",
9    "expected_position": "算法工程师"
10 }
```

岗位画像

代码块

```
1  {
2    "job_id": "J001",
3    "job_title": "算法工程师",
4    "required_skills": ["Python", "机器学习", "PyTorch"],
5    "preferred_skills": ["推荐系统", "Docker"],
6    "education_requirement": "本科",
```

```
7     "experience_requirement": 1,  
8     "city": "广州",  
9     "industry": "互联网"  
10 }
```

B 第4天：整理报告内容

B 最终提交：

代码块

1. 简历/JD需要抽取的字段
2. 抽取方法对比
3. 推荐采用的抽取方案
4. 技能标准化方法
5. 候选人画像JSON
6. 岗位画像JSON
7. 抽取风险与解决方案

C：推荐算法调研内容压缩版 用户492694

C 的4天核心任务

C 要回答：

我们用什么算法完成人岗匹配？如何证明算法有效？

C 第1天：确定推荐任务

建议不要做太多任务，主任务只定一个：

输入候选人画像，输出 Top-K 推荐岗位。

扩展任务可以写：

输入岗位，输出 Top-K 候选人。

但主线必须简单。

C 第2天：确定算法路线

建议采用“三层算法路线”。

第一层：规则匹配 baseline

根据技能、学历、经验、城市打分。

代码块

```
1  匹配分数 =
2  0.5 × 技能匹配度
3  + 0.2 × 学历匹配度
4  + 0.2 × 经验匹配度
5  + 0.1 × 城市匹配度
```

这是必须有的，因为简单、稳定、可解释。

第二层：文本语义匹配 baseline

调研：

代码块

```
1  TF-IDF / BM25
2  Sentence-BERT
```

建议结论：

使用 BM25 作为检索型 baseline，使用 SBERT 作为语义匹配 baseline。

第三层：知识图谱增强推荐

不要一开始承诺必须做 KGAT/KGCN。4天调研阶段建议写成：

代码块

```
1  基础实现：知识图谱路径打分
2  扩展实现：Node2Vec / LightGCN
3  挑战实现：KGAT / KGCN
```

最稳的图谱推荐方案是：

代码块

岗位与候选人共享技能越多，路径越多，图谱匹配分越高。

C 第3天：设计评价指标

如果有真实投递/点击数据：

代码块

- 1 Recall@K
- 2 Precision@K
- 3 NDCG@K
- 4 MRR
- 5 HitRate@K

如果没有真实行为数据：

代码块

- 1 人工标注准确率
- 2 F1-score
- 3 Top-K人工合理性评估
- 4 解释覆盖率

建议在报告中写：

若主数据集包含行为标签，则采用推荐系统指标；若数据缺少真实标签，则采用人工标注小样本进行验证。

C 第4天：整理报告内容

C 最终提交：

代码块

- 1 1. 推荐任务定义
- 2 2. Baseline算法对比
- 3 3. 知识图谱增强推荐方案
- 4 4. 综合匹配分数设计
- 5 5. 评价指标设计
- 6 6. 消融实验设计

D：系统与展示调研内容压缩版 无名氏

D 的4天核心任务

D 要回答：

这个项目最后怎么演示？系统长什么样？各模块怎么接起来？

D 第1天：确定系统功能

不要设计太复杂，核心功能先大概分为5个：这里可以看看能不能蹭一蹭LLM，有没有类似的已有的开源框架

代码块

1. 候选人画像展示
2. 岗位画像展示
3. Top-K岗位推荐
4. 推荐理由解释
5. 技能差距分析

扩展功能：

代码块

6. 知识图谱可视化
7. 算法指标对比
8. 岗位推荐候选人

D 第2天：确定技术架构

建议采用最稳方案：

代码块

- 1 前端：Streamlit
- 2 后端：FastAPI

- 3 数据库：SQLite / MySQL
- 4 图数据库：Neo4j
- 5 算法：Python模块调用
- 6 可视化：ECharts / PyVis / Neo4j Bloom

如果时间非常紧，甚至可以先不做完整前后端分离：

代码块

```
1 Streamlit + Python算法模块 + Neo4j
```

报告中可以写成：

原型系统采用 Streamlit 快速搭建，后续可扩展为 Vue + FastAPI 的前后端分离架构。

D 第3天：设计页面原型

至少设计4个页面：

页面	内容
首页	项目简介、系统流程
候选人页面	候选人技能、学历、项目经历
推荐页面	Top-K岗位、匹配分数
解释页面	匹配技能、缺失技能、图谱路径
实验页面	不同算法指标对比

页面原型可以用 PPT、draw.io、Figma、墨刀，不一定要开发。

D 第4天：整理报告内容

D 最终提交：

代码块

```
1 1. 系统功能设计
2 2. 系统架构图
3 3. 前后端技术选型
```

- 4 4. API接口设计
- 5 5. 页面原型
- 6 6. 答辩展示流程
- 7 7. 系统风险与备选方案

每人有空就维护一个自己的文档

四人最终交付物要求

每个人不要写太散，最终都按这个格式交：

代码块

- 1 1. 本模块目标
- 2 2. 调研内容
- 3 3. 可选方案对比
- 4 4. 推荐采用方案
- 5 5. 输入与输出
- 6 6. 与其他成员的接口
- 7 7. 风险与备选方案
- 8 8. 后续开发计划

推荐的最终技术路线

你们4天调研后，建议统一成下面这条路线：

代码块

- 1 数据层：
- 2 公开人岗匹配数据集 + 简历/ JD样例数据
- 3
- 4 画像层：
- 5 规则 + 技能词典抽取简历和岗位要求
- 6
- 7 图谱层：
- 8 Candidate - Skill - Job 为核心的知识图谱
- 9
- 10 推荐层：
- 11 规则匹配 + BM25 + SBERT + 知识图谱路径打分
- 12

- 13 解释层：
- 14 匹配技能 + 缺失技能 + 图谱路径
- 15
- 16 展示层：
- 17 Streamlit + Neo4j/PyVis 可视化

这条路线的优点是：**4人分工清楚、工程闭环完整、实现风险低、展示效果好。**

最小调研成果标准

4天结束后，至少应该拿到这些东西：

成果	负责人
数据源对比表	A
知识图谱 Schema 图	A
简历/JD抽取字 段表	B
画像 JSON 样例	B
推荐算法对比表	C
匹配分数公式与 评价指标	C
系统架构图	D
页面原型图	D
最终整合报告	全体
汇报PPT大纲	D主导，全体补 充

我建议你们调研时明确写出的项目定位

可以直接采用下面这段作为报告中的项目定位：

本项目面向人岗智能匹配场景，构建候选人、岗位、技能、行业、城市等实体组成的知识图谱，并结合简历/JD文本语义匹配方法，实现岗位推荐与匹配解释。系统首先通过规则和技能词典对简历与岗位描述进行结构化抽取，形成候选人画像和岗位画像；随后利用规则匹配、BM25、SBERT和知识图谱路径打分完成Top-K岗位推荐；最后通过匹配技能、缺失技能和图谱路径为推荐结果提供可解释分析。

这个定位比较稳，不会过度承诺 KGAT/KGCN 这种高风险模型，同时保留了扩展空间。